Experiment 1

Student Name: Yash Sharma
Branch: B.E. CSE
Semester: 5th
Subject Name: Full Stack

UID: 24BCS11074
Section/Group: 610-B
Date of Performance: July 2026
Subject Code: 24CSP-304

Easy

1. **Aim:** Create a Vite React app with a functional component that renders a student profile card using JSX, displaying name, course, and age

2.Objective:

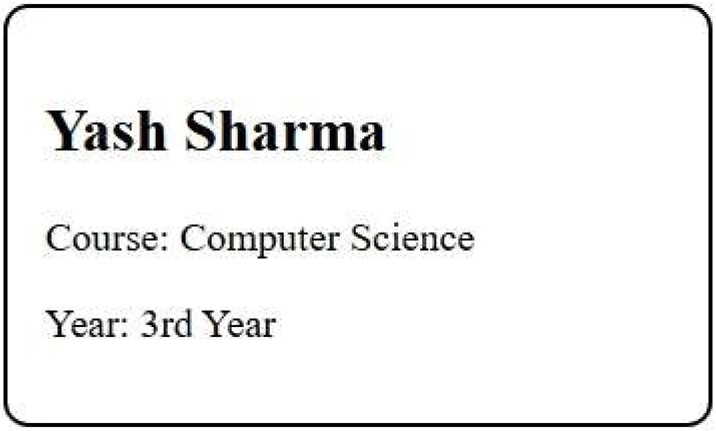
- Create a functional component in React using Vite.
- Render a static student profile card with JSX.
- Apply inline CSS styling for presentation.

3.Code:

- **StudentProfile.jsx**

```
function StudentProfile() {
  return (
    <div
      style={{
        border: "2px solid black",
        padding: "15px",
        width: "250px",
        borderRadius: "10px",
      }}
    >

      <h2>Yash Sharma </h2>
      <p>Course: Computer Science</p>
      <p>Year: 3rd Year</p>
    </div>
  );
}

export default StudentProfile;
```

4. Output:



5. Learning Outcome:

- Understanding functional components in React.
- Ability to use JSX for structuring UI elements.
- Applying inline styles to customize component appearance.
- Confidence in rendering static data inside a component.
- Foundation for reusable UI components in future tasks.

Medium

1. **Aim:** Build a reusable StudentCard component accepting props (name, age, course, grade). Render a list of 5 students dynamically using .map() with unique key props.

2.Objective:

- Understanding props as a way to pass data between components.
- Ability to design reusable components for structured data display.
- Practical use of .map() to iterate over arrays and render lists.
- Importance of unique keys in React for list rendering.
- Confidence in building dynamic UIs that scale with data size.

3.Code:

- **StudentCard.jsx**

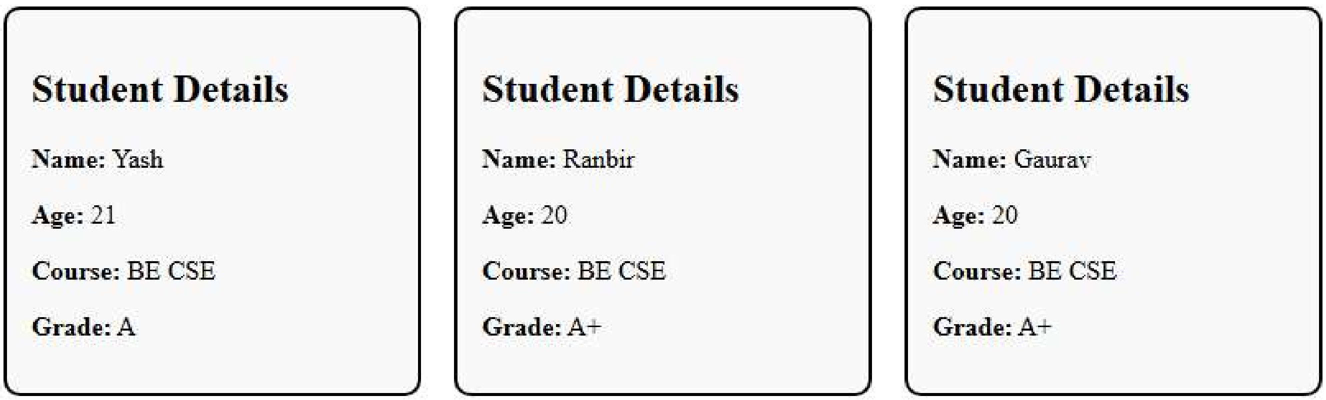
```
function StudentCard(props) {
  return (
    <div className="card">
      <h2>Student Details</h2>
      <p><b>Name:</b> {props.name}</p>
      <p><b>Age:</b> {props.age}</p>
      <p><b>Course:</b> {props.course}</p>
      <p><b>Grade:</b> {props.grade}</p>
    </div>
  );
}

export default StudentCard;
```

- **App.jsx**

```
import StudentCard from "./StudentCard";
import "./App.css";

function App() {
  const students = [
    { id: 1, name: "Yash", age: 21, course: "BE CSE", grade: "A" },
    { id: 2, name: "Ranbir", age: 20, course: "BE CSE", grade: "A+" },
    { id: 3, name: "Gaurav", age: 20, course: "BE CSE", grade: "A+" }
  ];

  return (
    <div className="container">
      {students.map((student) => (

        <StudentCard
```

```
          key={student.id}
          name={student.name}
          age={student.age}

          course={student.course}
          grade={student.grade}
        />
      ))}
    </div>
  );
}

export default App;
```

4. Output:



5. Learning Outcome:

- Understanding props as a way to pass data between components.
- Ability to design reusable components for structured data display.
- Practical use of .map() to iterate over arrays and render lists.
- Importance of unique keys in React for list rendering.
- Confidence in building dynamic UIs that scale with data size.

Hard

1. **Aim:** Develop a multi-section academic dashboard with nested components (Header, Sidebar, StudentList, Footer), passing data through props and using conditional rendering to show/hide sections.

2.Objective:

- Develop a multi-section academic dashboard with nested components.
- Pass data through props across multiple components.
- Use conditional rendering to show/hide sections dynamically.
- Apply external CSS for layout and styling.

3.Code:

- **Header.jsx**

```
function Header(props) {
  return (
    <header className="header">
      <h1>{props.title}</h1>
    </header>
  );
}

export default Header;
```

- **Footer.jsx**

```
function Footer() {

  return (

    <footer className="footer">

      <h3>© 2026 Chandigarh University</h3>

    </footer>

  );

}



export default Footer;
```

- **App.css**

```css
body {
  margin: 0;
  font-family: Arial;
}

.header {
  background: #1565c0;

  color: white;
  padding: 15px;
  text-align: center;
}

.sidebar {
  background: #eeeeee;
  padding: 15px;
  width: 200px;
  float: left;
  height: 250px;
}

.studentlist {
  margin-left: 220px;
  padding: 20px;
}
```

```
.footer {
  background: #1565c0;
  color: white;
  text-align: center;
  padding: 10px;
  clear: both;
}
```

- **App.jsx**

```
import Header from "./Header";
import Sidebar from "./Sidebar";
import StudentList from "./StudentList";
import Footer from "./Footer";
import "./App.css";

function App() {
  const students = [
    { id: 1, name: "Yash", course: "BE CSE" },
    { id: 2, name: "Ranbir", course: "BE CSE" },
    { id: 3, name: "Gaurav", course: "BE CSE" }
  ];

  const showStudents = true;

  return (
    <div>
      <Header title="Academic Dashboard" />
      <Sidebar />
```

```
      {showStudents ? (
        <StudentList students={students} />
      ) : (
        <h2>No Students Available</h2>
      )}
      <Footer />
    </div>
  );
}


export default App;
```

- **StudentList.jsx**

```
function StudentList(props) {
  return (
    <div className="studentlist">
      <h2>Student List</h2>
      {props.students.map((student) => (
        <p key={student.id}>
          {student.name} - {student.course}
        </p>
      ))}
    </div>
  );
}

export default StudentList;
```

- **Sidebar.jsx**

```
function Sidebar() {
  return (
    <div className="sidebar">
      <h3>Menu</h3>
      <ul>
        <li>Home</li>
        <li>Students</li>
        <li>Courses</li>
        <li>Results</li>
      </ul>
    </div>
  );
}

export default Sidebar;
```

4. Output:

5. Learning Outcome:

- Ability to structure applications with multiple nested components.
- Skill in conditional rendering for dynamic UI control.
- Understanding component hierarchy (Header, Sidebar, StudentList, Footer).
- Confidence in managing props across components.
- Capability to design scalable dashboards with clean separation of concerns.s